

ZYVORAI LABS · CUSTOMER DOCUMENTATION

Ephemera

Page-by-Page Product Manual

2026-08-29

Page-by-page guides

Each guide follows: Purpose → When to use it → How to get there → What you can do → Related pages.

Every route is also listed in the [complete page index](#).

Admin

Page	What it covers
Admin Basics	Deploy, systemd, ports, host prep, and operations.

Deploy

Page	What it covers
Kubernetes Deployment	DaemonSet / DisposableVm CRD operator path.

Images

Page	What it covers
Build Custom Images	Build and customize guest OS images for disposable VMs.

Integrations

Page	What it covers
Ragnarok Integration	Drive Ephemera disposable VMs from Ragnarok UI + SSO.

Onboarding

Page	What it covers
Getting Started	Install Ephemera and boot your first disposable VM.
Use Cases	Concrete scenarios — CI runners, golden images, fleets, sandboxes.

Operations

Page	What it covers
Workflows	Day-to-day create / exec / pause / TTL / warm-pool jobs.

Setup

Page	What it covers
Configuration	Backends, storage, auth, policy, and agent settings.

8 guides. Regenerate: `node scripts/customer-docs/generate-guide-index.mjs` .

Admin Basics

Purpose

Deploy, systemd, ports, host prep, and operations.

How to get there

- Topic id: `admin-basics`
- Section: **Admin** → **Admin Basics**

Guide

Topic	Guidance
Service	Run <code>ephemera serve</code> under the provided systemd unit; the TTL reaper and pool backfill only run while <code>serve</code> is up
Logs	<code>journalctl -u ephemera -f</code> ; each VM also has its own console log under <code><state_dir>/instances/<uuid>/console.log</code>
State	<code><state_dir>/vms.json</code> is the source of truth, coordinated across concurrent <code>ephemera</code> processes via an OS-level <code>flock</code> on <code>vms.lock</code>
Security	Bearer-token auth/RBAC is opt-in — configure <code>[[auth.tokens]]</code> before exposing the REST API beyond localhost; <code>extra_args</code> on a create request is an administrator escape hatch, never expose it to untrusted callers
Support	GitHub issues · Contact Zyvor for Enterprise

See also [Getting started](#) and [Configuration](#).

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Kubernetes Deployment

Purpose

DaemonSet / DisposableVm CRD operator path.

How to get there

- Topic id: `kubernetes-deployment`
- Section: **Deploy → Kubernetes Deployment**

Guide

`ephemera-kube` — the `DisposableVm` custom resource plus its node-local operator — ships as a proper container image and a set of `deploy/k8s/` manifests, so it can run as an actual per-node DaemonSet instead of only as a host-native systemd service.

This is a different model from a Pod-per-workload platform: a `DisposableVm` never goes through `RuntimeClass` or a container runtime shim. Each object maps directly to a raw VM process on whichever node its `spec.node` names, driven by that node's own `ephemera-kube` instance talking to a *local* `ephemera serve` REST API. There is no scheduler — placement is always explicit.

What gets deployed

One pod per capable node, two containers sharing the pod's network namespace:

Container	Runs	Privileges
<code>ephemera</code>	<code>ephemera serve</code> — the VMM control plane	<code>/dev/kvm</code> , <code>NET_ADMIN</code> / <code>SYS_ADMIN</code> / <code>NET_BIND_SERVICE</code> / <code>NET_RAW</code> , <code>hostNetwork: true</code>
<code>ephemera-kube</code>	The <code>DisposableVm</code> reconcile loop	None — <code>runAsNonRoot</code> , read-only root filesystem

Node prerequisites

Before labeling a node `ragnarok.io/ephemera-capable=true` (or your own equivalent label):

1. `/dev/kvm` present and accessible.
2. `nbd` kernel module loaded (`modprobe nbd`) — needed by the image-customization pipeline; this can't be satisfied from inside a container, it's a host-level step.
3. VM images pre-staged under the `state_dir` hostPath (there's no k8s-native image pull path yet — images must already exist at the path a `DisposableVm` 's `image` field names).

Deploy order

```
kubectl apply -f deploy/k8s/namespace.yaml
kubectl apply -f deploy/k8s/crd.yaml
kubectl apply -f deploy/k8s/rbac.yaml
kubectl apply -f deploy/k8s/configmap.yaml
kubectl label node <node-name> ragnarok.io/ephemera-capable=true
kubectl apply -f deploy/k8s/daemonset.yaml
```

`crd.yaml` is generated straight from the Rust type (`cargo run -p ephemera-kube -- --print-crd`), not hand-maintained — regenerate it whenever the CRD's Rust definition changes.

Verifying it

```
kubectl -n ephemera-system get pods -o wide  
kubectl -n ephemera-system logs ds/ephemera-kube -c ephemera-kube --follow
```

Then create a test `DisposableVm` targeting the labeled node and watch `phase` move from `Pending` to `Running` with a real `vmId` populated — see `deploy/k8s/README.md` 's smoke test for the full manifest. Deleting the object is finalizer-gated: `kubectl delete` blocks until the real VM is torn down, so a brief pause there is expected, not a hang.

Known limitations

- **Networking:** only `networkMode: none / user` are wired through the CRD today (NAT/port-forward), even though the underlying VMM backends already support TAP and macvtap. Bridged networking needs a CRD field addition plus a decision about how the DaemonSet's `hostNetwork: true` pod interacts with your cluster's CNI — not something to improvise per-cluster.
- **No scheduler:** whatever creates `DisposableVm` objects — e.g. [Ragnarok](#), which surfaces these as a parallel **Ephemera VMs** workload type alongside its KubeVirt VMs and Kata containers — is responsible for picking a concrete, capable node.
- **No image distribution:** images must be staged identically on every capable node by whatever process labels it.

Ragnarok product path

1. Deploy Ephemera CRD + DaemonSet (this page).
2. Install [Ragnarok](#) (KubeVirt required for the main fleet; Ephemera is opt-in).
3. Optional: wire Ragnarok OIDC/SSO (`--with-oidc`) — identity stays on Ragnarok; Ephemera does not terminate browser SSO.
4. Open Ragnarok → [Ephemera VMs](#). Missing operator → explicit banner, not a silent empty page.

Customer manuals: [Ephemera](#) · [Ragnarok](#).

Next steps

- [Admin basics](#)
- [Configuration](#)
- [Use cases](#)

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Build Custom Images

Purpose

Build and customize guest OS images for disposable VMs.

How to get there

- Topic id: `build-image-tutorial`
- Section: **Images** → **Build Custom Images**

Guide

`ephemera build-image` takes a base disk image and applies customizations — hostname, package installs, arbitrary commands, SSH-key injection, file copy-in, and systemd service enablement — to produce a new, ready-to-boot image. It does this through [GuestKit](#) mounting the image directly with `qemu-nbd` and running commands in a `chroot`. There's no `libguestfs` appliance and **no VM boot involved**, which means `build-image` doesn't need `/dev/kvm` at all — only root and the `nbd` kernel module.

This page walks through the three package-manager families `build-image` supports: Debian/Ubuntu (`apt`), RHEL-family (`dnf` / `tdnf` / `yum`), and Arch Linux (`pacman`). Every example here is real-hardware-verified and covered by an automated CI job that runs the same checks against fresh Ubuntu, Rocky Linux, and Arch cloud images on every commit — see [scripts/test-image-customize.sh](#) in the repo.

How it works, briefly

Field	What it does	Needs network?
<code>hostname</code>	Writes <code>/etc/hostname</code>	No
<code>packages</code>	Detects the guest's package manager, installs via it	Yes
<code>commands</code>	Runs each string via <code>sh -c</code> inside the chroot	Depends on the command
<code>ssh_key</code>	Appends to <code>/root/.ssh/authorized_keys</code> , <code>0600</code> perms	No
<code>copy_in</code>	Copies a host file into the image at the given path	No
<code>enable_services</code>	Runs <code>systemctl enable <name></code> for each unit	No

`packages` is the one field that needs real outbound networking — the guest's package manager has to actually reach its repositories. `build-image` handles this for you: it stages a working `/etc/resolv.conf` into the guest for the duration of the install (a stock cloud image's own `resolv.conf` is usually a dangling symlink that only resolves under a running systemd instance) and removes it again afterward.

Package-manager detection execs `command -v <tool>` inside the chroot and checks what's actually there, in this order: `apt-get` → `tdnf` → `dnf` → `yum` → `pacman`. If none are found, `packages` fails with a clear error telling you to use an equivalent `commands` entry instead.

Prerequisites

```
sudo modprobe nbd max_part=10
```

`scripts/bootstrap-host.sh` does this for you as part of general host setup. You need root (for the `qemu-nbd` mount) and enough free disk for a copy of the base image plus the output image — no VMM, no `/dev/kvm`.

Debian / Ubuntu

```
{
  "source": "/var/lib/ephemera/images/ubuntu-noble.qcow2",
  "output": "/var/lib/ephemera/images/ubuntu-dev.qcow2",
  "format": "qcow2",
  "hostname": "ubuntu-dev",
  "packages": ["tree", "jq", "qemu-guest-agent"],
  "commands": ["touch /etc/provisioned-by-ephemera"],
  "ssh_key": "ssh-ed25519 AAAAC3NzaC11ZDI1NTE5AAAA... you@example.com",
  "enable_services": ["qemu-guest-agent", "cron"]
}
```

```
sudo ephemera build-image --spec ubuntu-dev.json
```

Ubuntu's stock cloud image enables the `universe` component by default, so most common CLI tools install with no extra repo configuration. The system cron daemon's unit is `cron.service` — not `crond` (that's the RHEL-family name).

Rocky Linux, AlmaLinux, Fedora (`dnf`)

```
{
  "source": "/var/lib/ephemera/images/rocky9.qcow2",
  "output": "/var/lib/ephemera/images/rocky9-dev.qcow2",
  "format": "qcow2",
  "hostname": "rocky-dev",
  "packages": ["tree", "jq"],
  "commands": ["touch /etc/provisioned-by-ephemera"],
  "enable_services": ["crond"]
}
```

```
sudo ephemera build-image --spec rocky-dev.json
```

Rocky's `GenericCloud` image ships `cronie` (providing `crond.service`) pre-installed — `enable_services: ["crond"]` works without needing `packages` to install it first. Photon OS (`tdnf`) and older RHEL/CentOS 7 (`yum`) images go through the exact same code path.

Arch Linux (`pacman`)

```
{
  "source": "/var/lib/ephemera/images/arch.qcow2",
  "output": "/var/lib/ephemera/images/arch-dev.qcow2",
  "format": "qcow2",
  "hostname": "arch-dev",
  "packages": ["tree", "jq"],
  "commands": ["touch /etc/provisioned-by-ephemera"]
}
```

```
"enable_services": ["sshd"]
```

```
D
```

```
sudo ephemera build-image --spec arch-dev.json
```

Arch needs two things every other distro here doesn't — both handled for you automatically:

1. **Empty keyring.** A fresh Arch image ships with no trusted pacman keyring, so `build-image` runs `pacman-key --init` and `pacman-key --populate archlinux` before the actual install. This takes a few extra seconds the first time.
2. **No `/etc/mtab`.** `pacman` refuses to run without a readable `/etc/mtab` — on a real system that's a symlink to `/proc/self/mounts`, which doesn't exist in this bare chroot. `build-image` stages a minimal synthetic one for the duration of the install.

Arch's official cloud image doesn't ship cron by default, so this example enables `sshd` (which *is* preinstalled) instead — add `"cronie"` to `packages` and use `enable_services: ["cronie"]` if you want cron.

Verifying an image without booting it

Since `build-image` never boots a VM, you can sanity-check the output by mounting it directly, the same way it was built:

```
sudo modprobe nbd max_part=16
```

```
sudo qemu-nbd -c /dev/nbd0 /var/lib/ephemera/images/ubuntu-dev.qcow2
```

```
sudo partprobe /dev/nbd0 && sudo udevadm settle
```

```
sudo mount /dev/nbd0p1 /mnt # partition number varies by image layout
```

```
cat /mnt/etc/hostname
```

```
sudo umount /mnt
```

```
sudo qemu-nbd -d /dev/nbd0
```

Troubleshooting

- **"cannot install packages: no supported package manager found"** — the image's package manager isn't `apt/dnf/tdnf/yum/pacman` (e.g. Alpine's `apk`). Use `commands` to install packages manually.
- **A package install fails with a DNS/network-looking error** — check the *host's* own network/DNS first; `packages` needs the host to have real outbound connectivity.
- **`enable_services` fails with "unit does not exist"** — the service name is distro-specific (`cron` vs `crond`, `ssh` vs `sshd`), or the package providing it isn't installed yet.

Next steps

- [Common workflows](#)
- [Use cases](#)
- [Technical docs](#)

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Ragnarok Integration

Purpose

Drive Ephemera disposable VMs from Ragnarok UI + SSO.

How to get there

- Topic id: `ragnarok-integration`
- Section: **Integrations** → **Ragnarok Integration**

Guide

Ragnarok is the primary product UI/API that consumes Ephemera's Kubernetes path (`DisposableVm` + `ephemera-kube`). Ephemera stays a focused disposable-VM engine; Ragnarok is the operator console, RBAC, and SSO layer on top.

Roles

Layer	Owns
Ephemera	QEMU / Cloud Hypervisor / Firecracker VMs, TTL reaper, <code>ephemera serve</code> , <code>DisposableVm</code> CRD + node DaemonSet
Ragnarok	KubeVirt fleet + Ephemera Hub UI/API, JWT/OIDC/LDAP auth, RBAC, audit

Ragnarok never calls `ephemera serve` over the host REST API from the product path — it creates/reads/deletes `DisposableVm` CRs and lets the per-node operator talk to a **local** `ephemera serve` .

Install order (customer / lab)

0. **Optional — Ragnarok binary trial** (proprietary; Ephemera stays free):

```
VER=0.5
curl -LO "https://github.com/zyvorai/ephemera/releases/download/ragnarok-v${VER}/ragnarok-${VER}.tar.gz"
tar xzf "ragnarok-${VER}-linux-amd64.tar.gz" && cd "ragnarok-${VER}-linux-amd64"
ls -l trial.token # signed evaluation token – keep beside ./ragnarok
./install.sh
# edit ragnarok.env – then: set -a && source ragnarok.env && set +a && ./ragnarok
curl -s http://127.0.0.1:5010/api/v1/license/status
```

Or install Ragnarok via Helm / `deploy-remote.sh` from the private repo with `--set license.key=<jwt>` (same JWT as `trial.token`). After expiry: sales@zyvor.dev for a renewed token.

1. **Ephemera on capable nodes** — see `deploy/k8s/` in this repo (and [Kubernetes deployment](#) on the site):

```
kubectl apply -f deploy/k8s/namespace.yaml
kubectl apply -f deploy/k8s/crd.yaml
kubectl apply -f deploy/k8s/rbac.yaml
kubectl apply -f deploy/k8s/daemonset.yaml
kubectl label node <node> ragnarok.io/ephemera-capable=true
# Stage VM images under the DaemonSet state.dir hostPath on each node
```

2. **Ragnarok** — install KubeVirt first, then Ragnarok (Helm / Kustomize / `./scripts/deploy-remote.sh`). Docs:

- Technical: [zyvor.dev Ragnarok docs](#)
- Customer manual: [Ragnarok manual](#)
- OIDC/SSO: Ragnarok repo `docs/0IDC.md` and deploy `--with-oidc` (IdP proxy is Ragnarok's concern; Ephemera does not terminate SSO)

3. **Open Ragnarok UI** → **Ephemera VMs** (or Confidential / Ephemera Hub). If the operator is missing, Ragnarok shows an explicit "operator not detected" banner instead of a silent empty list.

What Ragnarok adds

- `GET /api/v1/ephemera/capability` — CRD present?
- `GET /api/v1/ephemera/nodes` — nodes labeled `ragnarok.io/ephemera-capable=true`
- CRUD under `/api/v1/ephemera/vms` — namespace-scoped to the caller's RBAC

No Ragnarok-specific CRD fields: anything you can `kubectl apply` as a `DisposableVm` , Ragnarok can create.

Customer manuals (published)

Product	Manual
Ephemera	https://zyvor.dev/docs/ephemera-manual
Ragnarok	https://zyvor.dev/docs/ragnarok-manual
Suite index	https://zyvor.dev/docs/customer-manuals

Auth note

Ephemera is free (Apache-2.0) — no Ragnarok license token applies to it.

SSO (Keycloak OIDC), local break-glass, LDAP, and the Ragnarok signed `trial.token` / commercial JWT live entirely in **Ragnarok** (proprietary). Ephemera's own REST API uses optional bearer tokens for direct `ephemera serve` callers; that is separate from the Ragnarok dashboard login and from Ragnarok trial tokens (`scripts/trial-tool.py` stays in the private Ragnarok repo only).

See also the longer integration notes in the [root README — Using Ephemera through Ragnarok](#).

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Getting Started

Purpose

Install Ephemera and boot your first disposable VM.

How to get there

- Topic id: `getting-started`
- Section: **Onboarding** → **Getting Started**

Guide

Ephemera is a Rust control plane for disposable VMs — it needs a Linux x86_64 host with `/dev/kvm` available, and at least one of `qemu-system-x86_64`, Cloud Hypervisor, or Firecracker installed.

Prerequisites

- A Linux x86_64 host with virtualization enabled and `/dev/kvm` present.
- A Rust toolchain (`cargo`) to build from source.
- `qemu-system-x86_64` and `qemu-img` at minimum; Cloud Hypervisor and Firecracker are optional per-backend.

1. Build and install

```
cargo build --release
sudo install -m 0755 target/release/ephemera /usr/local/bin/ephemera
sudo install -m 0644 config.example.toml /etc/ephemera.toml
```

A one-command host bootstrap script (`scripts/bootstrap-host.sh`) installs the system packages, Cloud Hypervisor, and Firecracker for you; `scripts/deploy-remote.sh` does the same over SSH to a remote host.

2. Create your first VM

```
ephemera create --spec examples/qemu.json
```

The response is the VM's full record — id, status, disk path, PID. It boots from a disposable copy-on-write overlay of the base image named in the spec, so the base image itself is never modified.

3. Run something inside it

```
ephemera exec <id> -- echo hello
```

This runs over the vsock guest agent — no SSH, no network path required at all, as long as the guest image has the agent installed and `agent.enabled: true` was set on the create request.

4. Let it clean up on its own

Set `ttl_seconds` on the create request and Ephemera's TTL reaper deletes the VM automatically once it expires — or delete it yourself:

```
ephemera delete <id>
```

Troubleshooting

- **create fails with "base image does not exist"** — the `image` path in the spec must exist on the host running `ephemera`, and (for the `ceph-rbd` / `lvm-thin` storage backends) follow their specific reference format — see [Configuration](#).
- **exec hangs or fails** — confirm the create request set `"agent": {"enabled": true}` and the guest image actually has `ephemera-guest-agent` installed and running.
- **/dev/kvm missing** — enable virtualization in the host's BIOS/hypervisor, and confirm the current user is in the `kvm` group or run as root.

Next steps

- [Configuration](#)
- [Common workflows](#)
- [Admin basics](#)

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Use Cases

Purpose

Concrete scenarios — CI runners, golden images, fleets, sandboxes.

How to get there

- Topic id: `use-cases`
- Section: **Onboarding** → **Use Cases**

Guide

Ephemera is a disposable-VM control plane: create a short-lived, isolated virtual machine backed by QEMU/KVM, Cloud Hypervisor, or Firecracker, use it, and let a TTL reaper clean it up. Every use case below maps directly onto what's implemented today — see the [technical docs](#) or the project's [README](#) for the full feature list.

Ephemeral CI/CD build and test runners

Spin up a real VM per job, run the job inside it over vsock `exec` (no SSH, no network path needed at all), and let `ttl_seconds` guarantee cleanup even if the job crashes or the runner disappears mid-job.

```
cat > ci-job.json <<'3504
{
  "name": "ci-job-4821",
  "backend": "firecracker",
  "image": "/var/lib/ephemera/images/ci-runner.raw",
  "vcpus": 2,
  "memory_mib": 2048,
  "network": {"mode": "none"},
  "ttl_seconds": 900,
  "agent": {"enabled": true, "port": 5000}
}

$ id
uid=0(root)

$ id=$(ephemera create --spec ci-job.json | jq -r .id)
$ ephemera exec "$id" -- ./run-tests.sh
```

Firecracker's jailer (chroot + uid/gid drop) gives each job its own privilege-dropped sandbox, and cgroup v2 resource control caps what a single job can consume on a shared runner host. `network.mode: "none"` plus vsock `exec` means a compromised or malicious test suite has no network path out at all.

Golden-image pipeline

Build a customized, versioned base image once — package installs, hostname, SSH keys, a baked-in agent binary — and reuse it across every VM you create from it, instead of provisioning each VM from scratch at boot time. See [Building custom OS images](#) for the full walkthrough across Debian/Ubuntu, RHEL-family, and Arch base images.

Pair it with the image catalog (SHA-256 + optional Ed25519 signing) to give every VM a provenance guarantee — a tenant references an image by name, and the daemon refuses anything that isn't a known, signed entry.

Kubernetes-native disposable workloads

For teams already running Kubernetes who want a real VM (not a container) for a specific workload — untrusted code, a kernel-dependent test, a legacy binary — the `DisposableVm` CRD plus the node-local `ephemera-kube` operator lets a VM be requested the same way any other Kubernetes resource is:



`kubectl delete disposablevm` blocks on a finalizer until the real VM is actually gone, and the operator self-heals — if the underlying VM disappears out-of-band, it gets replaced automatically without touching the CR. Verified against a real k3s cluster.

Multi-host fleets without Kubernetes

Not every team wants a Kubernetes control plane just to spread disposable VMs across a handful of bare-metal or edge hosts. `ephemera-agent` is a lighter-weight alternative: a central fleet registry plus a per-host heartbeat client, with load-aware placement deciding which host a new VM request lands on — verified across two real, physically separate hosts. This fits edge deployments, colo racks, or any fleet where standing up a full Kubernetes cluster is disproportionate to the workload.

Sandboxed / untrusted code execution

The combination that makes Ephemera suitable for running code you don't trust:

- **Firecracker jailer** — chroot + uid/gid drop, so even a process compromise doesn't hand over root on the host.
- **cgroup v2 resource control** — hard caps on CPU/memory/IO per VM.
- **Network namespaces** and `network.mode: "none"` — no network path out of the guest when the workload doesn't need one.
- **vsocx exec** — get output back from the guest without opening any network port, SSH included.
- **TTL reaper** — a VM that's forgotten about gets torn down anyway.

This is the same isolation shape used for malware-analysis sandboxes and "run this untrusted PR's code" CI steps, built from primitives this project already has.

Disposable dev/test environments

Give every branch, PR, or engineer their own real VM — not a shared staging box — with automatic cleanup via `ttl_seconds`. QEMU's qcow2 copy-on-write overlays mean spinning up a new VM from a golden image is cheap (no full disk copy), and `pause / resume` let you park an environment instead of destroying and rebuilding it.

Bring-your-own storage backend

Beyond the default qcow2/raw overlay, Ephemera supports LVM thin snapshots, NBD-exported disks, and Ceph RBD as storage backends — Ceph RBD verified against a real Rook Ceph cluster. This matters if you're deploying

into infrastructure that already standardized on one of these instead of adopting a new storage layer just for VM disks.

Networking that matches the environment

- **QEMU user-mode NAT** — zero host config, good for a single dev machine.
- **TAP + Linux bridge** — a VM on the same L2 as the host.
- **macvtap** — a VM's own MAC address directly on a parent link (no bridge).

All three are SSH-verified end-to-end in the project's own regression tests.

Next steps

- [Getting started](#)
- [Building custom OS images](#)
- [Common workflows](#)

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Workflows

Purpose

Day-to-day create / exec / pause / TTL / warm-pool jobs.

How to get there

- Topic id: `workflows`
- Section: **Operations** → **Workflows**

Guide

Workflow	Steps
CI/sandbox VM	<code>ephemera create</code> with <code>ttl_seconds</code> set → <code>ephemera exec</code> the build/test command → let the TTL reaper delete it, or <code>ephemera delete</code> explicitly
Warm pool for fast handoff	<code>ephemera pool create</code> with a template + size → <code>ephemera pool claim</code> hands back an already-booted, paused VM in roughly resume time
Multi-host fleet	Run <code>ephemera-agent central</code> once, <code>ephemera-agent node</code> on every host, then <code>POST /fleet/vms</code> with no target node for load-aware placement
Kubernetes-native	Apply the <code>DisposableVm</code> CRD, run <code>ephemera-kube</code> per node, then create/delete VMs as ordinary Kubernetes objects
Image build	<code>ephemera build-image --spec examples/build-image.json</code> — download, verify SHA-256, resize, and customize a base image once, reuse it for every VM. See Building custom OS images for a per-distro walkthrough (apt/dnf/pacman).

For full command references, see the [technical docs](#) and the project's own [README](#). See also: [Use cases](#).

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Configuration

Purpose

Backends, storage, auth, policy, and agent settings.

How to get there

- Topic id: `configuration`
- Section: **Setup → Configuration**

Guide

Ephemera is configured through one TOML file (`/etc/ephemera.toml` by default, or `--config <path>`), plus per-VM fields on each create request.

Backend and storage binaries

The `[global]` -level keys point at the VMM and storage tool binaries — `qemu_binary` , `qemu_img_binary` , `cloud_hypervisor_binary` , `ch_remote_binary` , `firecracker_binary` , and related paths. Defaults resolve from `$PATH` ; set them explicitly if a binary lives somewhere nonstandard.

Storage backends

Every create request may set `"storage"` to switch how that VM's disk is provisioned — the default (unset) is a qcow2 CoW overlay for QEMU or a reflinked raw file for Cloud Hypervisor/Firecracker, and needs no extra configuration:

Value	What it needs
<code>lvm-thin</code>	<code>image</code> must be a <code>/dev/<vg>/<lv></code> path to an existing LVM thin logical volume
<code>nbd</code>	QEMU only — no extra config; a <code>qemu-nbd</code> subprocess is spawned automatically
<code>ceph-rbd</code>	A <code>[storage]</code> section with <code>ceph_user</code> / <code>ceph_conf</code> ; <code>image</code> is a <code>pool/image</code> reference with a protected ephemera-base snapshot already on it

Isolation and resource control

- **Firecracker jailer** — a `[jailer]` section (`enabled` , `uid` , `gid` , `chroot_base_dir`) switches every Firecracker VM to launch through `jailer` instead of directly.
- **Network namespaces** — set `netns: true` on a request's `network` block to give that VM its own namespace instead of sharing the host bridge.
- **cgroup v2 resource control** applies automatically to every VM; use the REST API's `/resources` endpoint or the CLI to set CPU/memory/IO/pids/cpuset limits after creation.

Auth

An empty `[auth]` section (the default) leaves the REST API open — every request is treated as admin. Add one or more `[[auth.tokens]]` entries (`token` , `role: "admin" | "read-only"`) to require a bearer token on every request except `/healthz` .

Policy (admission limits)

An optional `[policy]` section caps `max_vcpus` , `max_memory_mib` , `max_disk_gib` , `max_ttl_seconds` , `allowed_backends` , and `allowed_image_dirs` — every field defaults to unrestricted.

Troubleshooting

- **A `storage=ceph-rbd` create fails immediately** — confirm `[storage].ceph_conf` points at a real `ceph.conf` , the referenced pool/image exists, and it has a protected snapshot named `ephemera-base` .
- **Auth returns 401 on every request** — a token is configured but wasn't sent, or doesn't match; requests need `Authorization: Bearer <token>` .
- **A policy-restricted create is rejected** — the response's error message names exactly which `[policy]` limit was exceeded.

Next steps

- [Getting started](#)
- [Common workflows](#)
- [Admin basics](#)

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Ephemera — Complete page index

Every primary navigable dashboard route.

Generated: 2026-08-29 · 8 routes

Regenerate: `node scripts/customer-docs/generate-page-index.mjs`

Onboarding

Page	Route	Purpose	Guide
Getting Started	<code>getting-started</code>	Install Ephemera and boot your first disposable VM.	Open
Use Cases	<code>use-cases</code>	Concrete scenarios — CI runners, golden images, fleets, sandboxes.	Open

Setup

Page	Route	Purpose	Guide
Configuration	<code>configuration</code>	Backends, storage, auth, policy, and agent settings.	Open

Operations

Page	Route	Purpose	Guide
Workflows	<code>workflows</code>	Day-to-day create / exec / pause / TTL / warm-pool jobs.	Open

Admin

Page	Route	Purpose	Guide
Admin Basics	<code>admin-basics</code>	Deploy, systemd, ports, host prep, and operations.	Open

Deploy

Page	Route	Purpose	Guide
Kubernetes Deployment	<code>kubernetes-deployment</code>	DaemonSet / DisposableVm CRD operator path.	Open

Images

Page	Route	Purpose	Guide
Build Custom Images	<code>build-image-tutorial</code>	Build and customize guest OS images for disposable VMs.	Open

Integrations

Page	Route	Purpose	Guide
Ragnarok Integration	<code>ragnarok-integration</code>	Drive Ephemera disposable VMs from Ragnarok UI + SSO.	Open

Related

- [Customer docs home](#)
- [Page-by-page guides](#)